

LAPORAN TUGAS BESAR 2
IF3270 PEMBELAJARAN MESIN
CONVOLUTIONAL NEURAL NETWORK DAN
RECURRENT NEURAL NETWORK



Disusun oleh Kelompok 62:

13523113	Kefas Kurnia Jonathan
13523118	Farrel Athalla Putra
13622076	Ziyan Agil Nur Ramadhan

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi.....	1
Daftar Tabel.....	2
Daftar Gambar.....	3
1. Deskripsi Persoalan.....	4
2. Penjelasan Implementasi dan Forward Propagation.....	6
2.1 Implementasi (Kelas, Atribut, dan Metode).....	6
2.2 Forward Propagation.....	12
2.2.1 Forward Propagation CNN.....	12
2.2.2 Forward Propagation Simple RNN.....	14
2.2.3 Forward Propagation LSTM.....	16
3. Hasil Pengujian.....	19
3.1 Hasil Pengujian CNN.....	19
3.1.1 Perbandingan shared vs non-shared parameter.....	19
3.1.2 Pengaruh jumlah layer konvolusi.....	20
3.1.3 Pengaruh banyak filter per layer konvolusi.....	21
3.1.4 Pengaruh ukuran filter per layer konvolusi.....	22
3.1.5 Pengaruh jenis pooling layer.....	22
3.1.6 Perbandingan Keras vs from scratch.....	23
3.2 Hasil Pengujian RNN.....	23
3.2.1 Perbandingan Jumlah Layer.....	24
3.2.2 Perbandingan Ukuran Hidden State.....	25
3.3 Hasil Pengujian LSTM.....	25
3.3.1 Perbandingan Jumlah Layer.....	25
3.3.2 Perbandingan Ukuran Hidden State.....	26
3.4 Hasil Pengujian RNN vs. LSTM.....	27
3.5 Perbandingan Keras dan from Scratch (RNN dan LSTM).....	30
3.6 Pengaruh panjang maksimum caption terhadap BLEU-4.....	31
4. Kesimpulan dan Saran.....	33
4.1 Kesimpulan.....	33
4.2 Saran.....	34
REFERENSI.....	35
LAMPIRAN.....	36
A. Pembagian Tugas.....	36
B. Form Penggunaan AI.....	36

Daftar Tabel

Tabel 1.1 Definisi Layer pada Modul CNN.....	4
Tabel 1.2 Definisi Layer pada Modul RNN dan LSTM.....	5
Tabel 3.1.1 Perbandingan jumlah parameter Conv2D vs LocallyConnected2D (arsitektur terbaik).....	19
Tabel 3.1.2 Macro F1 berdasarkan jumlah layer konvolusi.....	20
Tabel 3.1.3 Macro F1 berdasarkan kombinasi jumlah filter.....	21
Tabel 3.1.4 Macro F1 berdasarkan ukuran kernel filter.....	22
Tabel 3.1.5 Macro F1 berdasarkan jenis pooling layer.....	22
Tabel 3.1.6 Perbandingan macro F1 Keras vs from scratch pada test set.....	23
Tabel 3.2.1 Pengaruh jumlah layer terhadap performa SimpleRNN decoder.....	24
Tabel 3.3.1 Pengaruh jumlah layer terhadap performa LSTM decoder.....	25
Tabel 3.4.1 Perbandingan performa terbaik SimpleRNN vs LSTM.....	27
Tabel 3.4.2 Contoh caption dari 10 gambar representatif.....	28
Tabel 3.5.1 Perbandingan BLEU-4 dan waktu inferensi: Keras vs from scratch.....	30
Tabel 3.6.1 Pengaruh max_len terhadap BLEU-4 score.....	31

Daftar Gambar

Gambar 2.1 Operasi konvolusi 2D dengan shared parameters. Sumber : EpyNN.....	13
Gambar 3.1.1 Perbandingan kurva loss Conv2D vs LocallyConnected2D.....	20
Gambar 3.1.2 Perbandingan kurva loss berdasarkan jumlah layer konvolusi.....	21
Gambar 3.1.3 Pengaruh jumlah filter terhadap performa model CNN.....	21
Gambar 3.1.4 Pengaruh ukuran filter terhadap performa model CNN.....	22
Gambar 3.1.5 Perbandingan kurva loss MaxPooling vs AveragePooling.....	23
Gambar 3.2.1 Kurva loss SimpleRNN: variasi jumlah layer.....	24
Gambar 3.2.2 Pengaruh ukuran hidden state pada SimpleRNN.....	25
Gambar 3.3.1 Kurva loss LSTM: variasi jumlah layer.....	26
Gambar 3.3.2 Pengaruh ukuran hidden state pada LSTM.....	26
Gambar 3.4.1 Perbandingan BLEU-4 dan METEOR: SimpleRNN vs LSTM semua variasi.....	27
Gambar 3.4.2 Kurva loss semua variasi SimpleRNN.....	28
Gambar 3.4.3 Kurva loss semua variasi LSTM.....	28
Gambar 3.4.5 Sampel dengan caption RNN, LSTM, danground truth.....	29
Gambar 3.5.1 Perbandingan BLEU-4 dan waktu inferensi: Keras vs from scratch.....	30
Gambar 3.6.1 Pengaruh panjang maksimum caption terhadap BLEU-4 score.....	31

1. Deskripsi Persoalan

Pada zaman sekarang, perkembangan sistem *deep learning* seperti *Large Laguange Model* (LLM) multimodal dan sistem pengenalan suara, sangat bergantung pada fondasi arsitektur yang mampu mengekstraksi data spasial sekaligus memproses data sekuensial. Untuk memahami cara kerja sistem tersebut secara fundamental, diperlukan pemahaman teknis mengenai cara arsitektur penyusunnya bekerja dari level paling dasar. Oleh karena itu, pada Tugas Besar 2 IF3270 Pembelajaran Mesin, diberikan persoalan yang dibagi menjadi dua tahapan utama yaitu membangun model untuk klasifikasi citra menggunakan *Convulational Neural Network* (CNN) dan mengeksplorasi *pipeline image captioning* melalui arsitektur *encoder-decoder* yang menggabungkan ekstraksi fitur dari CNN dengan *Recurrent Neural Network* (RNN) atau *Long Short-Term Memory* (LSTM) sebagai pembangkit teks.

Analisis dan pemodelan akan menggunakan dua dataset yang berbeda untuk masing-masing tahap dengan dataset pertama adalah *Intel Image Classification* berukuran sekitar 25.000 gambar dengan 6 kategori serta dataset kedua adalah *Flickr8k* dengan 8.092 gambar dan 5 deskripsi untuk setiap gambar.

Pada pengerjaan tugas besar ini, algoritma CNN, Simple RNN dan LSTM akan diimplementasikan *from scratch* menggunakan bantuan *library* komputasi numerik seperti NumPy. Implementasi modul-modul algoritma ini harus memenuhi beberapa ketentuan antara lain.

1. Modul diimplementasikan secara modular dengan setiap *layer* memiliki *method forward* tersendiri yang menerima masukan dan mengembalikan keluaran berupa *numpy array*.
2. Modul harus dapat membaca dan memuat bobot serta bias dari model referensi yang telah dilatih sebelumnya menggunakan *library* Keras.
3. Pada eksperimen pembuatan model referensi Keras, fungsi *loss* yang digunakan wajib berupa *Sparse Categorical Crossentropy* dengan *Adam Optimizer*.
4. Untuk tahap *image captioning*, arsitektur menggunakan metode *pre-inject*, di mana vektor fitur gambar dari CNN dimasukkan di awal proses *sequence* sebelum token penanda urutan kata dimulai.

Modul CNN *from scratch* yang diimplementasikan harus mendukung beberapa jenis operasi spasial, dengan pilihan *layer* CNN yang akan diimplementasikan sebagai berikut:

Tabel 1.1 Definisi Layer pada Modul CNN

Nama Layer	Deskripsi
Conv2D	Operasi konvolusi spasial 2 dimensi dengan filter yang digeser

	secara berkala menggunakan konsep <i>shared parameters</i>
LocallyConnected2D	Operasi konvolusi 2 dimensi tanpa berbagi parameter dengan setiap posisi spasial memiliki bobot kernel yang independen.
Pooling (Max/Avg)	Reduksi dimensi spasial dengan mengambil nilai maksimal atau nilai rata-rata pada setiap channel menggunakan jendela geser.
Global Pooling (Max/Avg)	Reduksi seluruh area spasial menjadi satu nilai representatif per <i>channel</i> kemudian membuang dimensi tinggi dan lebar sepenuhnya.
Flatten	Mengubah bentuk matriks citra multi-dimensi menjadi vektor satu dimensi secara berurutan (row-major order).

Selain arsitektur visual, modul *decoder* untuk memproses dan menghasilkan teks juga wajib diimplementasikan dengan komponen berikut:

Tabel 1.2 Definisi Layer pada Modul RNN dan LSTM

Nama Layer	Deskripsi
Embedding Layer	Mengubah token kata berupa <i>integer</i> menjadi representasi vektor bernilai padat yang dapat dipelajari oleh model.
SimpleRNN Cell	Sel memori dasar yang memproses urutan teks dengan mempertahankan informasi jangka pendek melalui pembaruan hidden state.
LSTM Cell	Sel memori tingkat lanjut yang mampu mengendalikan informasi jangka panjang menggunakan arsitektur gerbang logika (gates) dan cell state.
Dense Layer	Digunakan ganda sebagai proyeksi fitur visual di awal arsitektur dan sebagai lapisan output akhir (menggunakan aktivasi Softmax) untuk menghasilkan distribusi probabilitas kosakata.

Sebagai tahap akhir pemodelan, hasil prediksi dari implementasi *from scratch* akan dievaluasi kinerjanya, dengan menggunakan *macro F1-score* untuk klasifikasi citra, serta *BLEU-4 score* dan *METEOR score* pada *image captioning*.

2. Penjelasan Implementasi dan Forward Propagation

2.1 Implementasi (Kelas, Atribut, dan Metode)

Implementasi terbagi pada 6 bagian utama, yaitu struktur *repository*, modul *shared* yang digunakan bersama, tiga modul model utama, dan modul berisikan bonus-bonus yang dibuat. Selengkapnya dapat dilihat sebagai berikut.

2.1.1 Struktur Repository

Kode implementasi dibangun dalam struktur repositori sebagai berikut.

```
CNN-RNN-digaspolndangakndangak/  
src/  
  shared/      # Utilitas bersama (image_utils, caption_utils, metrics)  
  cnn/         # Implementasi CNN from scratch  
    layers/    # Conv2D, LocallyConnected2D, Pooling, Flatten  
    model.py   # CNNScratch wrapper  
  rnn/         # Implementasi SimpleRNN from scratch  
    layers/    # Embedding, SimpleRNNCell, SimpleRNN  
    model.py   # RNNDecoder  
  lstm/        # Implementasi LSTM from scratch  
    layers/    # Embedding, LSTMCell, LSTM  
    model.py   # LSTMDecoder  
  bonus/       # Fitur bonus  
    gradcam.py  
    beam_search.py  
    init_inject.py  
    backprop/  
  notebooks/   # 13 notebook Jupyter  
  data/        # Dataset Intel dan Flickr8k  
  features/    # Cache fitur CNN  
  models/      # Model terlatih (.keras)  
  doc/         # Laporan
```

2.1.2 Modul *Shared*

Modul *shared* adalah modul yang digunakan bersama-sama untuk implementasi CNN, SimpleRNN dan juga LSTM. Beberapa modul tersebut di antaranya:

image_utils.py

Modul ini berisi tiga fungsi untuk *load* dan memproses gambar menggunakan PIL/Pillow dan NumPy:

- `load_image(path, target_size)`: Membuka gambar dari path yang diberikan menggunakan `PIL.Image.open`, dikonversi ke mode RGB, di-*resize* ke ukuran target, kemudian nilai pikselnya dinormalisasi ke rentang [0,1] dengan membagi dengan 255.
- `load_batch(paths, target_size)`: Memuat banyak gambar sekaligus dari daftar path. Hasilnya dikumpulkan menjadi satu NumPy array berukuran (N,H,W,C).
- `extract_and_cache_features(image_paths, keras_encoder, cache_path, batch_size, target_size)`: Mengekstrak fitur dari semua gambar menggunakan CNN encoder Keras yang

bobotnya sudah di-*freeze*, kemudian menyimpan hasilnya ke disk dalam format .npy. Jika file cache sudah tersedia, langsung dibaca ulang tanpa menghitung ulang.

caption_utils.py

Modul ini digunakan untuk *preprocessing caption*, yang berisikan:

- `clean_caption`: Mengubah teks ke huruf kecil dan menghapus tanda baca dengan `re.sub`.
- `build_vocabulary`: Membangun kamus kata dari seluruh caption data training, dengan token khusus `<pad>`, `<start>`, `<end>`, dan `<unk>`.
- `encode_caption`: Mengubah kata-kata menjadi indeks integer, menambahkan token khusus di awal dan akhir, kemudiannn melakukan padding atau truncation ke panjang yang seragam.

Konstanta `SPECIAL_TOKENS` menyimpan indeks tetap untuk keempat token khusus tersebut.

dense.py

Implementasi Dense layer yang digunakan ulang dari Tugas Besar 1, dengan atribut:

- `W`: matriks bobot ($n_{in} \times n_{out}$)
- `b`: vektor bias (n_{out})
- `activation`: pilihan fungsi aktivasi (linear, relu, softmax)

Method `forward(x)` menghitung `output = activation(xW + b)`. Input bisa satu sampel (1D) maupun batch (2D).

metrics.py

Berisi kumpulan fungsi evaluasi, meliputi:

- `macro_f1`: Wrapper dari `sklearn.metrics.f1_score` dengan `average='macro'`.
- `bleu4`: BLEU-4 score via `nltk.translate.bleu_score.corpus_bleu` dengan smoothing function `method1`.
- `meteor`: METEOR score via `nltk.translate.meteor_score`.

Method `forward(x)` untuk menghitung `output = activation(xW + b)`. Input bisa terdiri atas satu sampel (1D) maupun batch (2D).

2.1.3 Modul CNN

Terdapat beberapa kelas pada implementasi modul CNN, yang dapat dilihat sebagai berikut.

Kelas Conv2D

Atribut:

- `kernel`: np.ndarray shape ($k_H, k_W, C_{in}, C_{out}$)
- `bias`: np.ndarray shape (C_{out})
- `activation`: string ("relu" / "softmax" / "linear")

- `strides`: tuple (s_H, s_W), default (1,1)
- `padding`: string (“valid” / “same”)

Method:

- `load_weights(keras_layer)`: Membaca bobot dari layer Keras lewat `get_weights()` dan konfigurasi lewat `get_config()`.
- `forward(x)`: Forward pass untuk satu gambar (H, W, C).
- `forward_batch(X)`: Forward pass untuk batch (N, H, W, C).
- `_pad_input(x)`: Menambahkan padding ke input sesuai mode.
- `_apply_activation(z)`: Menerapkan fungsi aktivasi ke output.

Kelas **LocallyConnected2D**

Atribut:

- `kernel`: np.ndarray shape ($\text{out_rows} \times \text{out_cols}, k_H \cdot k_W \cdot C_{\text{in}}, C_{\text{out}}$)
- `bias`: np.ndarray shape ($\text{out_rows} \times \text{out_cols}, C_{\text{out}}$)
- `kernel_size`: tuple (k_H, k_W)
- `strides`: tuple (s_H, s_W)
- `output_shape_spatial`: tuple ($\text{out_rows}, \text{out_cols}$)

Method:

- `load_weights(keras_layer)`: Membaca bobot dan konfigurasi dari layer **LocallyConnected2D** Keras.
- `forward(x)`: Untuk setiap posisi (i, j), menghitung indeks posisi $p = i \cdot W_{\text{out}} + j$, kemudian mengalikan patch yang sudah di-*flatten* dengan `kernel[p]`.
- `forward_batch(X)`: Memanggil forward untuk setiap sampel dalam batch secara satu per satu karena bobot berbeda di tiap posisi sehingga tidak bisa divektorkan seperti **Conv2D** biasa.

Kelas **MaxPooling2D** dan **AveragePooling2D**

Atribut:

- `pool_size`: tuple (p_H, p_W)
- `strides`: tuple (s_H, s_W), default sama dengan `pool_size`

Method:

- `load_weights(keras_layer)`: Membaca konfigurasi dari Keras.
- `forward(x)`: Sliding window yang mengambil nilai max atau rata-rata di setiap window yang dilakukan per channel.
- `forward_batch(X)`: Versi batch dengan vektorisasi pada dimensi N

Kelas **GlobalMaxPooling2D** dan **GlobalAveragePooling2D**

Dua kelas ini mereduksi seluruh feature map spasial jadi satu nilai per channel dengan:

- `forward(x)`: `x.max(axis=(0,1))` atau `x.mean(axis=(0,1))`
- `forward_batch(X)`: `X.max(axis=(1,2))` atau `X.mean(axis=(1,2))`

Kelas Flatten

Me-*reshape* tensor (H,W,C) jadi vektor ($H \cdot W \cdot C$), dengan urutan row-major (C order), sesuai perilaku Keras: `x.flatten(order='C')`

Kelas MaxPooling2D dan AveragePooling2D

Atribut:

- `layers`: daftar layer from scratch yang disusun secara sekuensial

Method:

- `add(layer)`: Menambahkan layer baru ke ujung stack.
- `load_from_keras(keras_model)`: Iterasi semua layer Keras melewati layer yang tidak memiliki bobot (Input, Dropout, Rescaling), dan memanggil `load_weights` untuk yang memiliki bobot.
- `forward(x)`: Meneruskan satu sampel melalui semua layer satu per satu.
- `predict(x)`: Dapat menerima satu sampel (3D) maupun batch (4D). Untuk batch, akan digunakan `forward_batch` jika tersedia atau loop biasa jika tidak tersedia.

2.1.4 Modul SimpleRNN

Terdapat beberapa kelas pada implementasi modul SimpleRNN, yang dapat dilihat sebagai berikut.

Kelas SimpleRNNCell

Atribut:

- `cells`: daftar objek SimpleRNNCell, satu per layer

Method:

- `add_layer(cell)`: Menambahkan cell baru ke stack.
- `forward(x, return_sequences)`: Iterasi timestep demi timestep serta hidden state tiap layer diawali nol. Jika `return_sequences=True` mengembalikan semua hidden state (T, units); jika False hanya hidden state timestep terakhir.
- `forward_batch(X, return_sequences)`: Versi batch dengan input (N,T,input_dim).
- `forward_with_cache(x, return_sequences)`: Menyimpan semua `x_seq` dan `h_seq` per layer, dibutuhkan untuk backpropagation.

Kelas RNNDecoder

Atribut:

- `feature_projection`: Dense linier, mengubah `feature_dim` ke `embed_dim`
- `embedding`: layer Embedding
- `rnn`: SimpleRNN multi-layer
- `output_dense`: Dense softmax, dari `units` ke `vocab_size`
- `vocab`: dict (kata ke indeks)
- `idx2word`: dict (indeks ke kata)

Method:

- `load_weights(keras_model, vocab)`: Menelusuri layer Keras berdasarkan nama dan tipe, kemudian memuat bobotnya ke komponen yang sesuai.
- `generate_caption(feature_vec, max_len)`: Menghasilkan caption satu kata per langkah dengan greedy decoding, memakai arsitektur pre-inject.
- `generate_captions_batch(features, max_len, batch_size)`: Versi batch dari caption generation, semua sampel diproses bersamaan.

2.1.5 Modul LSTM

Terdapat beberapa kelas pada implementasi modul LSTM, yang dapat dilihat sebagai berikut.

Kelas LSTMCell

Atribut:

- `W_x`: np.ndarray shape (input_dim, 4 · units)
- `W_h`: np.ndarray shape (units, 4 · units)
- `b`: np.ndarray shape (4 · units,)
- `units`: jumlah unit hidden (integer)

Method:

- `load_weights(keras_layer)`: Membaca bobot Keras. Bias di Keras kadang berbentuk 2D [`bx`, `bh`]; jika bentuknya demikian, keduanya dijumlahkan: `b = bias[0] + bias[1]`.
- `forward(x_t, h_prev, c_prev)`: Menghitung keempat gate dan memperbarui state. Mengembalikan tuple (`ht`, `ct`).
- `forward_with_cache(x_t, h_prev, c_prev)`: Seperti forward tetapi juga mengembalikan `gates_dict` yang isinya nilai keempat gate, digunakan ketika backpropagation.
- `forward_batch(x_t, h_prev, c_prev)`: Versi batch.

Kelas LSTM

Atribut:

- `cells`: daftar LSTMCell

Method:

- `add_layer(cell)`: Menambahkan cell ke stack.
- `forward(x, return_sequences)`: Iterasi timestep, menjaga state ganda (h, c) per layer. Output bisa berupa seluruh sequence atau hanya state akhir.
- `forward_batch(X, return_sequences)`: Versi batch.
- `forward_with_cache(x, return_sequences)`: Menyimpan semua `x_seq`, `h_seq`, `c_seq`, `i/f/g/o_seq` per layer untuk backpropagation.

Kelas LSTMDecoder

Strukturnya sama dengan `RNNDecoder`, yang membedakan hanya modul recurrent-nya diganti LSTM. Karena LSTM memiliki dua state, cell state `c` juga diinisialisasi nol terpisah dari hidden state `h`.

2.1.6 Modul Bonus

Beberapa modul bonus yang diimplementasikan beserta dengan deskripsinya dapat dilihat pada poin-poin berikut.

Grad-CAM (gradcam.py)

Implementasi visualisasi Grad-CAM untuk melihat bagian mana dari gambar yang paling berpengaruh ke prediksi model:

- `grad_cam(keras_model, image, class_idx)`: Memakai `tf.GradientTape` untuk menghitung gradien output kelas terhadap aktivasi layer konvolusi terakhir.
- Hasilnya adalah heatmap.
- `get_intermediate_feature_maps(model, image)`: Mengambil aktivasi dari layer konvolusi tengah-tengah model.
- `overlay_heatmap(image, heatmap)`: Menumpuk heatmap di atas gambar asli dengan colormap dan transparansi tertentu.

Beam Search (beam_search.py)

Dataclass `BeamHypothesis`:

- `tokens`: daftar token yang sudah diprediksi
- `score`: log-probability kumulatif hipotesis ini
- `h_states`: hidden state per layer
- `c_states`: cell state per layer (khusus LSTM)

Dengan fungsi utama:

- `beam_search_rnn(decoder, feature_vec, vocab, idx2word, beam_size, max_len)`: Mengimplementasikan beam search untuk decoder RNN maupun LSTM. Pada setiap

langkah, top-k hipotesis dengan skor terbaik yang diteruskan. Caption terpilih dinormalisasi dengan panjangnya.

Grad-CAM (gradcam.py)

Arsitektur alternatif di mana fitur gambar digabung ke output RNN/LSTM, bukan dimasukkan di awal sebagai input:

- `build_init_inject_rnn()`: Membangun model Keras yang memakai layer Add antara feature projection dan output hidden state RNN.
- `build_init_inject_lstm()`: Versi LSTM dari arsitektur di atas.
- `train_init_inject(model, dataset, val_dataset, model_name)`: Melatih model dengan masked loss dan ModelCheckpoint.

Backpropagation (bonus/backprop/)

Penulis juga mengimplementasikan bonus Backpropagation dengan beberapa file berikut:

- `conv2d_backward.py`: Menghitung gradien (d_x , d_{kernel} , d_{bias}) untuk Conv2D melalui double-loop konvolusi.
- `rnn_backward.py`: Backpropagation through time (BPTT) untuk SimpleRNN. Gradien hidden state merambat balik sebagai $\nabla h_{t-1} = \nabla h_t \cdot W_h^T$, dengan turunan $\tanh(1 - h_t^2)$.
- `lstm_backward.py`: BPTT untuk LSTM. Lebih rumit karena ada gradien dari keempat gate ditambah gradien cell state yang mengalir mundur lewat forget gate.

2.2 Forward Propagation

2.2.1 Forward Propagation CNN

Forward propagation pada Convolutional Neural Network (CNN) merupakan proses menghitung *output* prediksi dari sebuah gambar input dengan melewatkannya secara berurutan melalui setiap *layer* pada jaringan. Setiap layer memiliki fungsi khusus, seperti mengekstraksi fitur visual, mereduksi dimensi, hingga menghasilkan probabilitas kelas. Pada implementasi ini, *layer-layer* yang digunakan meliputi Convolutional Layer, Locally Connected Layer, Pooling Layer, Flatten Layer, Fully Connected Layer, dan Activation Layer.

Operasi Konvolusi 2D (Shared Parameters)

Convolutional layer merupakan komponen utama CNN yang bertugas mengekstraksi pola lokal dari citra, seperti tepi, tekstur, dan bentuk. Layer ini menggunakan sejumlah *kernel* yang digeser ke seluruh area gambar. Pada setiap posisi, nilai pada *patch* input dikalikan dengan bobot *kernel*, kemudian dijumlahkan untuk menghasilkan satu nilai *output*. Apabila *input* x berukuran (H, W, C_{in}) dan *kernel* K berukuran $(k_H, k_W, C_{in}, C_{out})$, maka *output* konvolusinya adalah

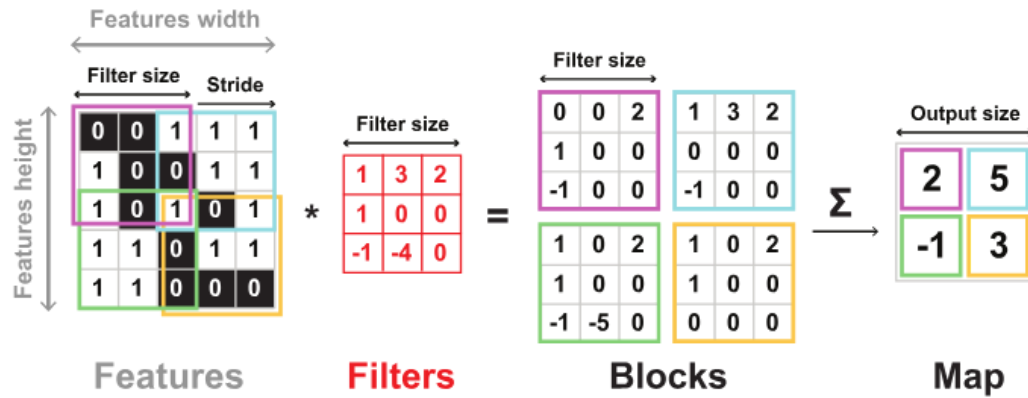
$$out[i, j, k] = \sum_{p=0}^{k_H-1} \sum_{q=0}^{k_W-1} \sum_{c=0}^{C_{in}-1} x[i \cdot s_H + p, j \cdot s_W + q, c] \cdot K[p, q, c, k] + b[k] \quad (2.1)$$

dihitung untuk setiap posisi *output* (i, j) dan setiap *filter* k . Salah satu karakteristik utama *convolutional layer* adalah *shared parameters*, yaitu satu *filter* yang sama digunakan pada seluruh posisi *input*. Dengan demikian, model dapat mendeteksi pola yang sama di berbagai lokasi gambar dengan jumlah parameter yang relatif sedikit. Dimensi *output* berupa:

$$H_{out} = \lfloor \frac{H_{pad} - k_H}{s_H} \rfloor + 1 \quad (2.2)$$

$$W_{out} = \lfloor \frac{W_{pad} - k_W}{s_W} \rfloor + 1 \quad (2.3)$$

Untuk *padding* yang sama, ditambahkan $p_H = \lfloor k_H/2 \rfloor$ baris dan $p_W = \lfloor k_W/2 \rfloor$ kolom di setiap sisi agar ukuran *output* tetap sama dengan *input* yang berasumsi nilai *stride* adalah 1.



Gambar 2.1 Operasi konvolusi 2D dengan *shared parameters*. Sumber : [EpyNN](#)

Locally Connected 2D (Non-Shared Parameters)

Locally Connected Layer memiliki prinsip kerja yang mirip dengan *convolutional layer*, yaitu menggunakan *patch* lokal untuk menghasilkan *output*. Perbedaannya terletak pada bobot yang digunakan. Pada layer ini, setiap posisi output memiliki kernel yang berbeda, sehingga parameter tidak dibagikan antar lokasi. Bobot berukuran $(H_{out} \cdot W_{out}, k_H \cdot k_W \cdot C_{in}, C_{out})$, dan *forward pass*-nya berupa

$$out[i, j, :] = flatten(x[i : i + k_H, j : j + k_W, :]) \cdot K[i \cdot W_{out} + j] + b[i \cdot W_{out} + j] \quad (2.4)$$

Pooling Layer

Pooling layer digunakan untuk mereduksi dimensi spasial *feature map* tanpa mengubah jumlah *channel*. Tujuan utama *pooling* adalah mengurangi kompleksitas komputasi, menurunkan risiko *overfitting*, dan meningkatkan ketahanan terhadap pergeseran kecil pada input. Berikut beberapa metode *pooling* yang digunakan:

1. **MaxPooling2D** : memilih nilai maksimum pada setiap jendela *pooling*. Operasi ini mempertahankan fitur yang paling dominan.

$$out[i, j, c] = \max_{p, q} x[is_H + p, js_W + q, c]$$

2. **AveragePooling2D**: menghitung rata-rata seluruh nilai di dalam jendela *pooling*.

$$out[i, j, c] = \frac{1}{p_H p_W} \sum_{p, q} x[is_H + p, js_W + q, c]$$

3. **GlobalMaxPooling2D**: mengambil nilai maksimum dari seluruh posisi pada setiap *channel*.

$$out[c] = \max_{h, w} x[h, w, c]$$

4. **GlobalAveragePooling2D**: menghitung rata-rata seluruh nilai pada setiap *channel*.

$$out[c] = \frac{1}{HW} \sum_{h, w} x[h, w, c]$$

Flatten Layer

Setelah proses ekstraksi fitur selesai, tensor (H, W, C) di-*reshape* ke vektor $(H \cdot W \cdot C)$ dalam urutan *row-major*, sehingga $flatten[h \cdot W \cdot C + w \cdot C + c] = x[h, w, c]$. Proses ini tidak mengubah nilai data, melainkan hanya mengubah bentuk representasinya di memori.

Aktivasi

Activation function memperkenalkan non-linearitas ke dalam jaringan sehingga model dapat mempelajari hubungan yang kompleks. Fungsi aktivasi yang digunakan berupa:

$$ReLU(z) = \max(0, z) \quad (2.5)$$

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (2.6)$$

Fully Connected Layer

Fully connected layer menghitung kombinasi linear dari seluruh elemen input. Jika input berupa vektor $x \in R^d$, maka output dihitung dengan:

$$y = xW + b \quad (2.7)$$

dengan $W \in R^{d \times u}$ dan $b \in R^u$. Layer ini digunakan pada bagian akhir jaringan untuk menghasilkan representasi yang dapat diklasifikasikan.

2.2.2 Forward Propagation Simple RNN

Forward propagation pada *Simple Recurrent Neural Network* (RNN) merupakan proses mengolah data sekuensial secara bertahap dari satu langkah waktu (timestep) ke langkah waktu berikutnya. Ini berbeda dengan CNN yang memproses data secara independen, sedangkan RNN memiliki memori internal berupa *hidden state* yang berfungsi untuk menyimpan konteks informasi dari *timestep* sebelumnya. Dalam konteks pemodelan *image captioning* ini, RNN bertugas sebagai *decoder* yang membangkitkan teks deskripsi gambar kata demi kata.

Persamaan SimpleRNN Cell

Pada setiap timestep t , sebuah sel SimpleRNN memproses dua informasi utama secara bersamaan, yaitu input pada saat ini ($x_t \in R^d$) dan *hidden state* dari langkah sebelumnya ($h_{t-1} \in R^u$). Kedua masukan ini dikalikan dengan matriks bobot nya masing-masing, dijumlahkan dengan nilai bias, lalu dilewatkan pada fungsi aktivasi *non-linear Hyperbolic Tangent* (tanh). Proses ini menghasilkan *hidden state* baru yang dirumuskan sebagai:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

di mana $W_x \in R^{d \times u}$ adalah matriks bobot untuk input, $W_h \in R^{u \times u}$ adalah matriks bobot untuk *hidden state* sebelumnya, dan $b \in R^u$ merupakan vektor bias. Pembaruan *state* inilah yang memungkinkan model untuk mengignat urutan kata dalam suatu kalimat.

Multi-Layer SimpleRNN

Untuk mendeteksi pola teks yang lebih kompleks, beberapa sel SimpleRNN dapat ditumpuk membentuk arsitektur *multi-layer*. Pada susunan ini, *hidden state* yang dihasilkan oleh *layer* ke- l pada timestep t akan langsung diteruskan menjadi input untuk *layer* di atasnya (*layer* ke- $l + 1$) pada timestep yang sama. Persamaannya secara matematis adalah:

$$h_t^{(l)} = \tanh(h_t^{(l-1)} W_x^{(l)} + h_{t-1}^{(l)} W_h^{(l)} + b^{(l)})$$

dengan ketentuan bahwa $h_t^{(0)}$ adalah masukan awal x_t yang bentuknya bisa berupa representasi *embedding* kata ataupun hasil proyeksi dari fitur gambar.

Arsitektur Pre-Inject untuk Image Captioning

Model ini menggunakan pendekatan *pre-inject* untuk menggabungkan informasi visual dan teks. Fitur gambar $v \in R^{2048}$ yang sebelumnya telah diekstraksi oleh CNN tidak langsung disambungkan dengan teks, melainkan diproyeksikan terlebih dahulu ke ruang dimensi *embedding* menggunakan sebuah *dense layer*:

$$x_{-1} = v W_{proj} + b_{proj}$$

Vektor hasil proyeksi $x_{-1} \in R^{de}$ ini kemudian disisipkan pada urutan paling awal sebagai $t = -1$ ke dalam decoder sebelum urutan teks dimulai. Dengan demikian, sekuens masukan utuhnya akan berbentuk $[x_{-1}, emb(< start >), emb(S_0), ..., emb(S_{N-1})]$. Saat proses dimulai pada *timestep* pertama, seluruh *hidden state* diinisialisasi dengan matriks nol ($h_0 = 0$) untuk semua *layer*.

Greedy Decoding

Pada tahap pembangkitan *caption*, model yang dikembangkan menggunakan pendekatan *greedy decoding* untuk memprediksi satu kata terbaik pada setiap langkahnya. Alur kerjanya dimulai dengan memasukkan vektor fitur gambar x_{-1} untuk memperbarui *hidden state* dari nilai nol. Selanjutnya, token penanda awal kalimat dimasukkan ke dalam layer *embedding* dan diteruskan menyusuri semua layer RNN.

Hidden state yang keluar dari layer RNN paling akhir kemudian diolah oleh *dense output layer* untuk menghasilkan nilai *logit* atau distribusi probabilitas untuk seluruh kosakata. Kata dengan nilai probabilitas tertinggi (*argmax*) akan dipilih sebagai prediksi untuk timestep tersebut. Kata hasil prediksi ini selanjutnya dijadikan input untuk timestep berikutnya. Siklus perulangan ini terus dilakukan hingga model memprediksi token penanda akhir kalimat, atau jika panjang teks telah mencapai batas maksimal yang ditentukan.

2.2.3 Forward Propagation LSTM

Long Short-Term Memory (LSTM) merupakan pengembangan dari arsitektur Simple RNN yang dirancang secara khusus untuk mengatasi permasalahan gradien hilang (*vanishing gradient*) dan mampu mengingat informasi dalam sekuens yang jauh lebih panjang. Kemampuan ini dicapai dengan memperkenalkan komponen memori tambahan yang disebut cell state (c_t), beserta serangkaian mekanisme gerbang (*gates*) yang berfungsi menyaring, menambah, atau membuang informasi secara selektif.

Persamaan LSTM Cell

Pada setiap langkah waktu atau timestep t , sebuah sel LSTM menerima tiga buah masukan, yaitu input pada saat itu ($x_t \in R^d$), *hidden state* dari timestep sebelumnya ($h_{t-1} \in R^u$), dan *cell state* dari timestep sebelumnya ($c_{t-1} \in R^u$). Proses perhitungannya diawali dengan menghitung kombinasi linear dari masukan x_t dan h_{t-1} yang menghasilkan matriks $g \in R^{4u}$:

$$g = x_t W_x + h_{t-1} W_h + b$$

Matriks g ini kemudian dipecah menjadi empat segmen yang masing-masing berukuran u . Setiap segmen dilewatkan pada fungsi aktivasi tertentu untuk membentuk empat gerbang utama LSTM:

1. Input Gate (i_t): $i_t = \sigma(g[0:u])$, berfungsi menentukan seberapa banyak informasi atau memori baru yang relevan untuk disimpan.

2. Forget Gate (f_t): $i_t = \sigma(g[u: 2u])$, bertugas mengontrol informasi lama mana dari *cell state* sebelumnya yang sudah tidak relevan dan harus dibuang.
3. Candidate Cell ($\bar{c}_t$): $\bar{c}_t = \tanh(g[2u: 3u])$, merupakan kandidat nilai memori baru yang siap ditambahkan ke dalam *cell state*.
4. Output Gate (o_t): $o_t = \sigma(g[3u: 4u])$, menentukan bagian mana dari *cell state* saat ini yang akan dikeluarkan sebagai *hidden state* baru.

Setelah keempat gerbang tersebut dihitung, model akan memperbarui *cell state* (c_t) dan *hidden state* (h_t) menggunakan operasi perkalian elemen per elemen ($\odot$):

$$\bar{c}_t = f_t \odot c_{t-1} + i_t \odot \bar{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

Pembaruan cell state ini bertindak seperti *conveyor belt*. Mekanisme ini memungkinkan informasi penting mengalir ke langkah-langkah waktu selanjutnya tanpa banyak mengalami perubahan linier, sehingga konteks memori jangka panjang tetap terjaga dengan baik.

Format Bobot Keras

Dalam mengimplementasikan modul *forward propagation* yang memuat bobot dari *library* Keras, terdapat format struktur penyimpanan yang perlu diperhatikan. Keras menyusun matriks bobot LSTM dengan urutan gabungan gerbang input, forget, *candidate/cell* (g), dan *output* (atau [i, f, g, o]), di mana masing-masing gerbang berukuran u . Oleh karena itu, ukuran total matriks bobot input (W_h) adalah $d \times 4u$ sedangkan matriks bobot recurrent (W_h) berukuran $u \times 4u$. Terkait dengan nilai bias, Keras terkadang menyimpannya dalam format array 2D berukuran (2, 4u) yang merupakan pasangan terpisah antara bias input (b_x) dan bias recurrent (b_h). Pada implementasi *from scratch* ini, kedua vektor bias tersebut cukup dijumlahkan menjadi satu buah vektor 1D berukuran $4u$.

Arsitektur Pre-Inject untuk LSTM

Penerapan LSTM pada arsitektur *pre-inject* untuk *image captioning* memiliki alur kerja yang pada dasarnya identik dengan Simple RNN. Vektor fitur visual tetap diproyeksikan dan disisipkan sebagai input pada timestep $t = -1$ sebelum token awal teks dimulai. Perbedaan utamanya hanya terletak pada manajemen memori awalnya. Selain harus menginisialisasi hidden state dengan matriks nol ($h_0 = 0$), arsitektur berlapis pada LSTM juga mewajibkan

setiap layer untuk menginisialisasi cell state masing-masing dengan nilai nol ($c_0 = 0$) pada awal proses dekode teks.

3. Hasil Pengujian

Berikut merupakan hasil pengujian yang mencakup beberapa hal berikut.

- CNN:
 - Perbandingan shared vs non-shared parameter
 - Pengaruh jumlah layer konvolusi
 - Pengaruh banyak filter per layer konvolusi
 - Pengaruh ukuran filter per layer konvolusi
 - Pengaruh jenis pooling layer
 - Perbandingan Keras vs from scratch
- Simple RNN dan LSTM untuk Image Captioning
 - Perbandingan jumlah layer: RNN
 - Perbandingan jumlah layer: LSTM
 - Perbandingan ukuran hidden state: RNN
 - Perbandingan ukuran hidden state: LSTM
 - Perbandingan RNN vs LSTM
 - Perbandingan Keras vs from scratch
 - Pengaruh panjang maksimum caption terhadap BLEU-4

3.1 Hasil Pengujian CNN

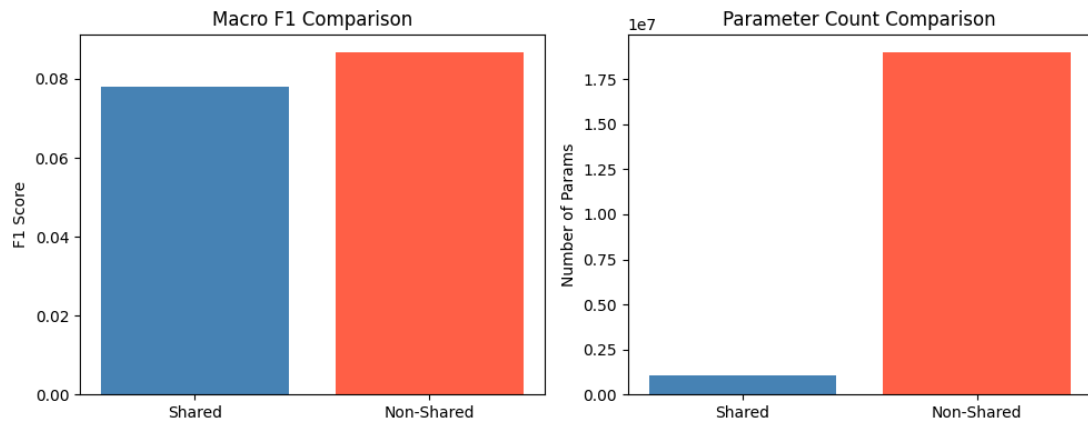
Berdasarkan hasil implementasi kode CNN dan run notebook, berikut adalah hasil yang didapatkan.

3.1.1 Perbandingan shared vs non-shared parameter

Pada Conv2D, satu kernel berukuran (kH, kW, Cin, Cout) dipakai bersama di semua posisi output sehingga total parameter kernel hanya $kH \times kW \times Cin \times Cout$. Di LocallyConnected2D, setiap posisi memiliki kernelnya masing-masing. Total parameternya menjadi $Hout \times Wout \times kH \times kW \times Cin \times Cout$. Dengan $Hout = Wout = 148$ (input 150×150 , kernel 3×3 , valid padding), jumlah parameternya sekitar $148 \times 148 \approx 21.900$ kali lebih banyak per layer dibanding Conv2D.

Tabel 3.1.1 Perbandingan jumlah parameter Conv2D vs LocallyConnected2D (arsitektur terbaik)

Arsitektur	F1	Parameter Total
Shared (Conv2D)	0.078043	1062918
Non-Shared (LocallyConnected2D)	0.086821	19008454



Gambar 3.1.1 Perbandingan kurva loss Conv2D vs LocallyConnected2D

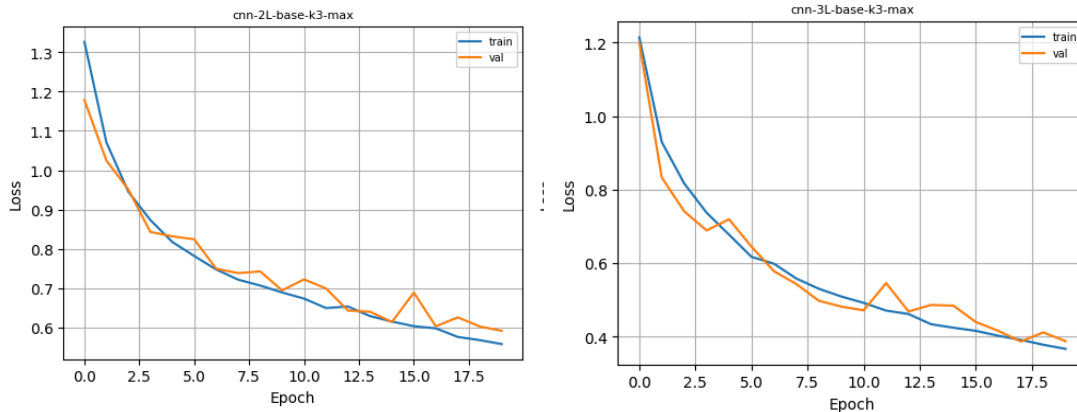
LocallyConnected2D memiliki kapasitas lebih besar karena parameternya jauh lebih banyak. Tetapi tanpa parameter sharing, model tidak mempelajari fitur yang bersifat translationinvariant, yaitu kemampuan mengenali pola yang sama di lokasi berbeda. Dampaknya ada tiga, yaitu: waktu pelatihan menjadi jauh lebih lama, risiko overfitting meningkat terutama pada dataset berukuran sedang, serta performa akhirnya tidak pasti lebih baik dari Conv2D meski memiliki parameter yang lebih banyak.

3.1.2 Pengaruh jumlah layer konvolusi

Jumlah Layer Konvolusi, dengan menambahkan kedalaman model dari 2 layer jadi 3 layer memberikan dampak yang paling terlihat, di mana seluruh model yang menggunakan 3 layer skor Macro F1-nya stabil di atas 0.83, berbeda lumayan jauh dengan yang hanya menggunakan 2 layer. Ini menunjukkan jika model membutuhkan layer lebih dalam untuk mengenali fitur gambar yang lebih kompleks.

Tabel 3.1.2 Macro F1 berdasarkan jumlah layer konvolusi

Jumlah Layer	Filter	Kernel	Pooling Best	val_loss	Macro F1
2	[32, 64]	3	max	0.5914	0.7860
3	[32, 64, 128]	3	max	0.3864	0.8626



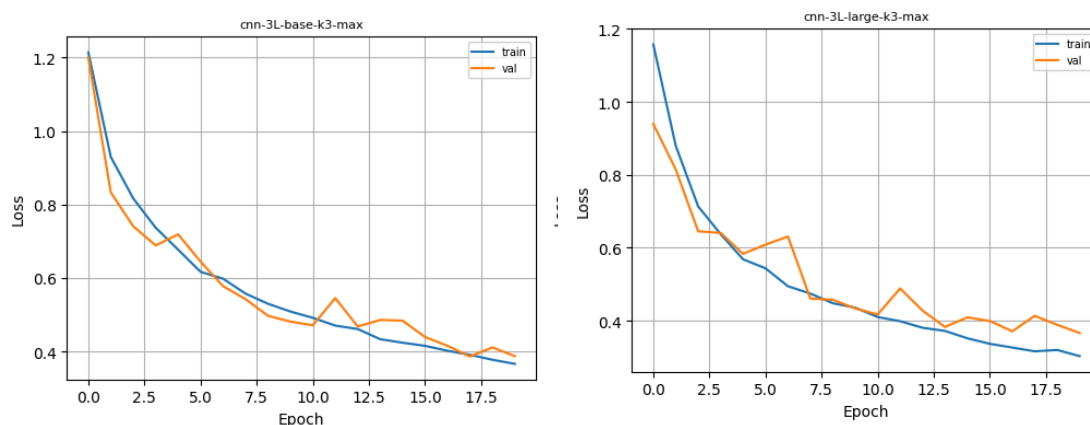
Gambar 3.1.2 Perbandingan kurva loss berdasarkan jumlah layer konvolusi

Lebih banyak layer konvolusi membuat model dapat mempelajari fitur yang lebih abstrak secara bertahap. Layer awal biasanya mendeteksi hal-hal sederhana seperti tepi atau warna, sedangkan layer yang lebih dalam menangkap pola-pola yang lebih kompleks seperti tekstur atau bentuk objek. Pada eksperimen ini, model dengan 3 layer diharapkan dapat menghasilkan representasi yang lebih kaya, meski di sisi lain risiko overfitting juga naik seiring bertambahnya parameter.

3.1.3 Pengaruh banyak filter per layer konvolusi

Tabel 3.1.3 Macro F1 berdasarkan kombinasi jumlah filter

Jumlah Layer	Filter	Kernel	PoolingBest	val_loss	Macro F1
3	[32, 64, 128]	3	max	0.3864	0.8626
3	[64, 128, 356]	3	max	0.3665	0.8707



Gambar 3.1.3 Pengaruh jumlah filter terhadap performa model CNN

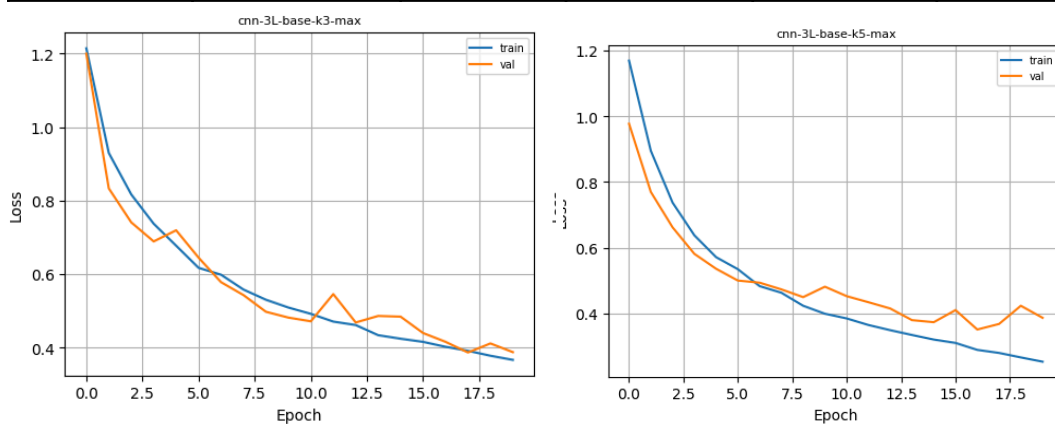
Jumlah filter menentukan seberapa lebar representasi yang dipelajari di tiap layer. Dalam eksperimen ini, konfigurasi large ([64,128,256]) secara umum mengalahkan konfigurasi dasar ([32,64,128]) meski selisihnya tidak terlalu besar (0.8707 vs 0.8626 pada 3L, k3, Max). Filter

yang lebih banyak memang dapat menangkap lebih banyak jenis pola. Tetapi di sisi lain, parameter akan bertambah dan risiko overfitting meningkat. Hasilnya akan sangat bergantung pada ukuran dataset.

3.1.4 Pengaruh ukuran filter per layer konvolusi

Tabel 3.1.4 Macro F1 berdasarkan ukuran kernel filter

Jumlah Layer	Filter	Kernel	PoolingBest	val_loss	Macro F1
3	[32, 64, 128]	3	max	0.3864	0.8626
3	[32, 64, 128]	5	max	0.3511	0.8782



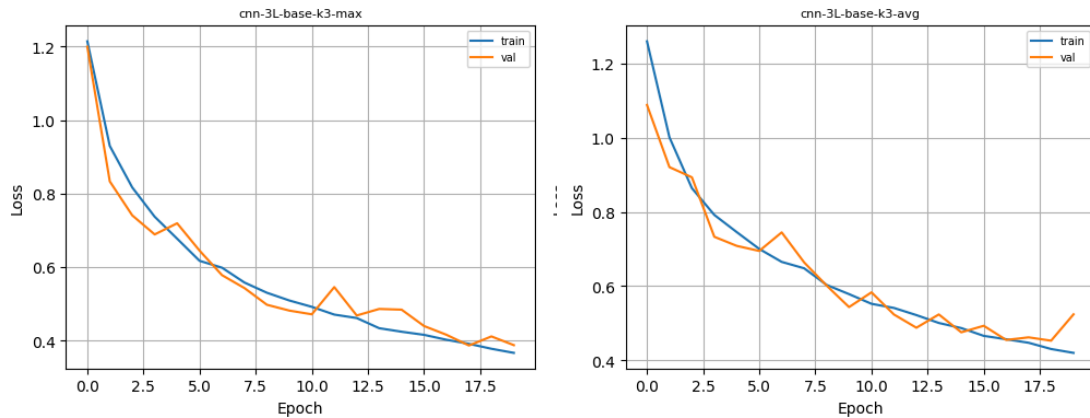
Gambar 3.1.4 Pengaruh ukuran filter terhadap performa model CNN

Ukuran filter menentukan seberapa luas area gambar yang dilihat setiap neuron konvolusi dalam sekali operasi. Filter 3×3 cukup populer karena cakupannya seimbang antara konteks lokal dan jumlah parameter. Filter 5×5 memberikan pandangan yang lebih luas sehingga cocok untuk gambar yang cukup besar seperti Intel Image Classification (150×150 piksel) dengan pola spasial objek tersebar di area yang lebih lebar. Pada eksperimen ini, 5×5 secara konsisten lebih baik 3×3 di semua konfigurasi lainnya. Hal ini sejalan dengan karakteristik dataset tersebut.

3.1.5 Pengaruh jenis pooling layer

Tabel 3.1.5 Macro F1 berdasarkan jenis pooling layer

Jumlah Layer	Filter	Kernel	PoolingBest	val_loss	Macro F1
3	[32, 64, 128]	3	max	0.3864	0.8626
3	[32, 64, 128]	3	avg	0.4532	0.8334



Gambar 3.1.5 Perbandingan kurva loss MaxPooling vs AveragePooling

MaxPooling mengambil nilai terbesar dari setiap window sehingga fitur yang paling mencolok lebih terjaga. Hal ini juga membuat model lebih tahan terhadap pergeseran kecil pada gambar. Sementara itu, AveragePooling mengambil nilai rata-rata dan menghasilkan representasi yang lebih mulus tapi cenderung membuang informasi fitur yang tajam. Untuk klasifikasi gambar, MaxPooling biasanya lebih cocok karena mempertahankan fitur-fitur yang paling diskriminatif.

3.1.6 Perbandingan Keras vs from scratch

Untuk memvalidasi implementasi, penulis membandingkan output forward propagation from scratch dengan output Keras pada data test.

Tabel 3.1.6 Perbandingan macro F1 Keras vs from scratch pada test set

Implementasi	Macro F1
Keras	0.8829
From Scratch	0.8829

Evaluasi menunjukkan agreement rate sebesar 100% antara Keras dan from scratch. Artinya, untuk semua 3.000 gambar test, prediksi kelas dari kedua implementasi benar-benar identik. Karena bobot dimuat langsung dari model yang sama, hal ini seharusnya memang terjadi kecuali ada perbedaan urutan operasi numerik. Makro F1 sebesar 0.8829 berasal dari evaluasi model. Perbedaan waktu inferensi tidak diukur secara eksplisit karena implementasi from scratch menggunakan loop Python murni yang jauh lebih lambat dari kernel TensorFlow yang ter-compile.

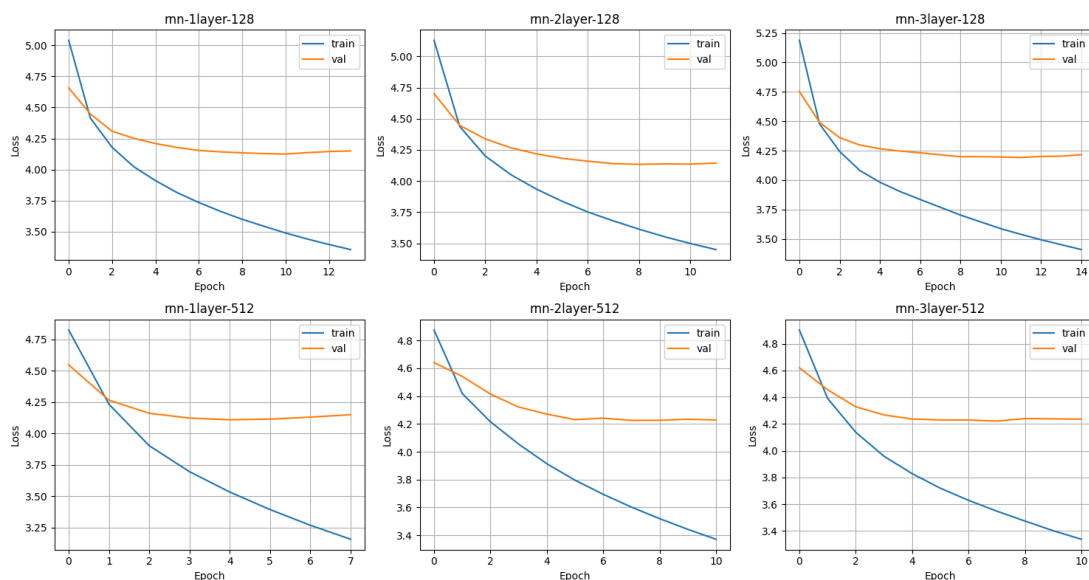
3.2 Hasil Pengujian RNN

Berdasarkan hasil implementasi kode RNN dan run notebook, berikut adalah hasil yang didapatkan.

3.2.1 Perbandingan Jumlah Layer

Tabel 3.2.1 Pengaruh jumlah layer terhadap performa SimpleRNN decoder

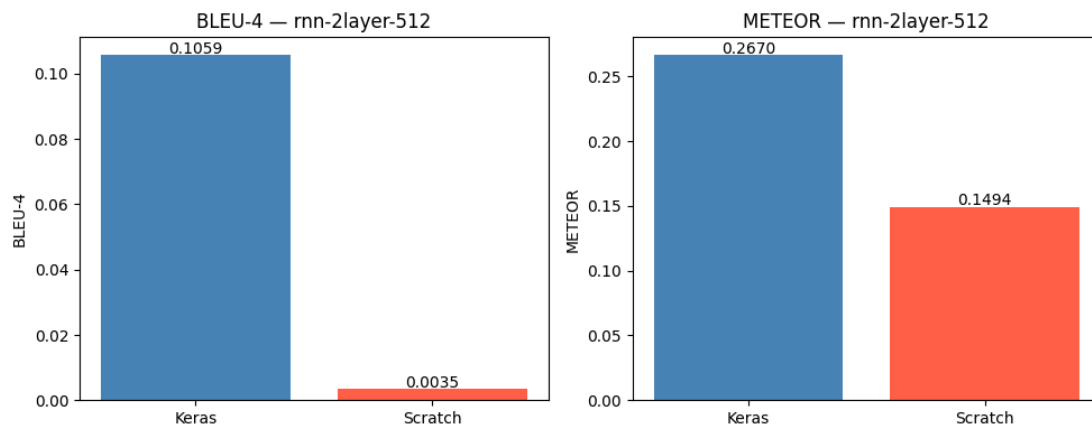
Layer	Units	BLEU-4	METEOR	Waktu Latih (s)	Param.
1	128	0.0747	0.2326	770	≈3.5M
1	512	0.0888	0.2431	1259	≈6.8M
2	128	0.0544	0.2189	689	≈3.6M
2	512	0.1059	0.2670	2078	≈7.4M
3	128	0.0588	0.2192	934	≈3.6M
3	512	0.0899	0.2428	2389	≈7.9M



Gambar 3.2.1 Kurva loss SimpleRNN: variasi jumlah layer

Menambah layer recurrent membuat model bisa menangkap pola sekuensial yang lebih dalam. Tetapi di SimpleRNN, masalah vanishing gradient bisa semakin parah jika layer-nya terlalu banyak, terutama saat caption yang harus dihasilkan cukup panjang. Dengan hidden size kecil seperti 128, kapasitas model memang terbatas dan kemungkinan sulit menghafal informasi gambar sepanjang proses decoding.

3.2.2 Perbandingan Ukuran Hidden State



Gambar 3.2.2 Pengaruh ukuran hidden state pada SimpleRNN

Hidden state yang lebih besar memberi model lebih banyak ruang untuk menyimpan informasi. Dalam konteks image captioning, model harus tetap "ingat" tentang isi gambar sambil menghasilkan kata satu per satu. Dalam hal ini, kapasitas yang cukup memang penting. Meski begitu, hidden size 512 juga membawa konsekuensi: pelatihan lebih lambat dan ada risiko overfitting pada dataset yang tidak terlalu besar.

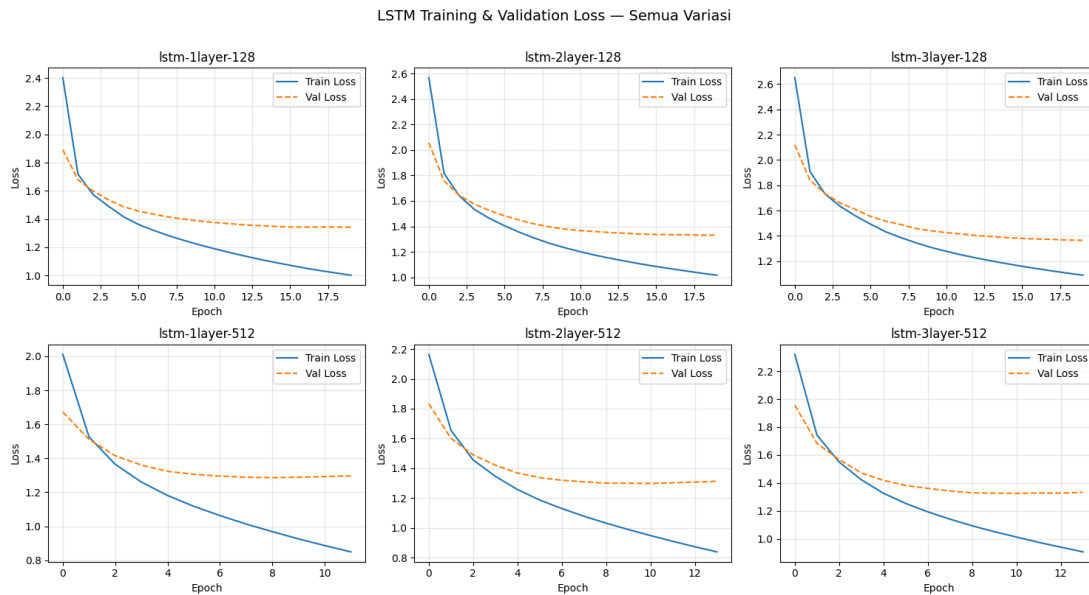
3.3 Hasil Pengujian LSTM

Berdasarkan hasil implementasi kode LSTM dan run notebook, berikut adalah hasil yang didapatkan.

3.3.1 Perbandingan Jumlah Layer

Tabel 3.3.1 Pengaruh jumlah layer terhadap performa LSTM decoder

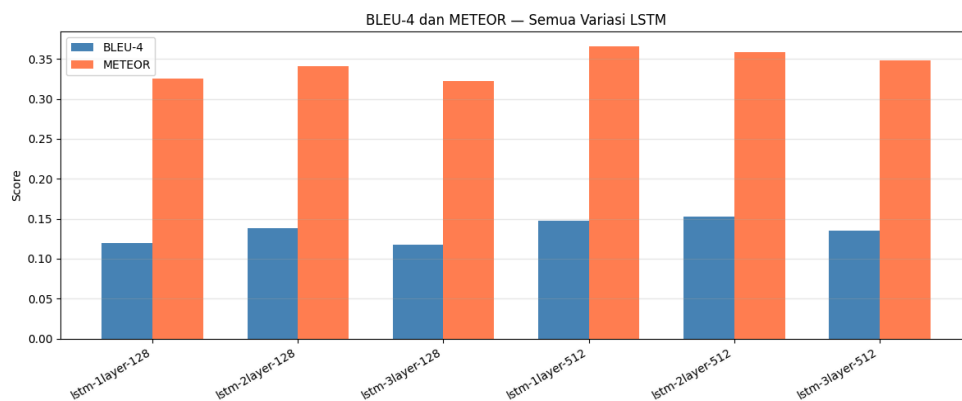
Layer	Units	BLEU-4	METEOR	Waktu Latih (s)	Param.
1	128	0.1200	0.3261	314.3	≈3.7M
1	512	0.1447	0.3636	477.6	≈8.0M
2	128	0.1375	0.3393	481.0	≈3.8M
2	512	0.1519	0.3577	546.0	≈10.1M
3	128	0.1176	0.3227	824.3	≈3.9M
3	512	0.1347	0.3474	777.3	≈12.2M



Gambar 3.3.1 Kurva loss LSTM: variasi jumlah layer

LSTM memiliki mekanisme forget gate dan cell state yang membantu gradien tetap mengalir meskipun sequence-nya panjang. Oleh karena itu, masalah vanishing gradient tidak separah pada SimpleRNN. Hasilnya berupa penambahan layer di LSTM cenderung tetap memberikan perbaikan performa serta tidak langsung memburuk seperti yang sering terjadi di SimpleRNN berlapis.

3.3.2 Perbandingan Ukuran Hidden State



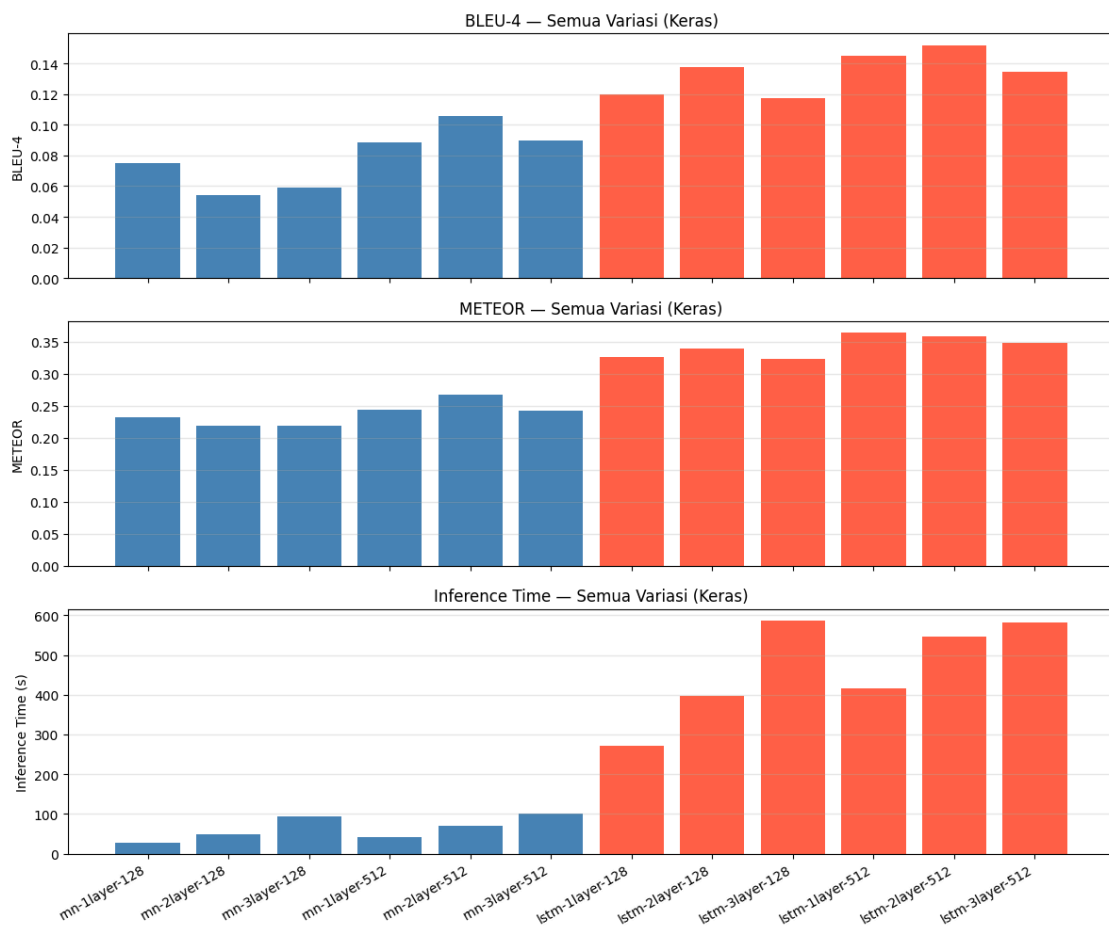
Gambar 3.3.2 Pengaruh ukuran hidden state pada LSTM

Pada LSTM, efek hidden size yang lebih besar cenderung lebih terasa dibanding SimpleRNN. Hal ini masuk akal karena LSTM memiliki mekanisme gating yang secara aktif memilih informasi mana yang disimpan dan mana yang dilupakan sehingga kapasitas tambahan bisa dimanfaatkan dengan lebih efektif.

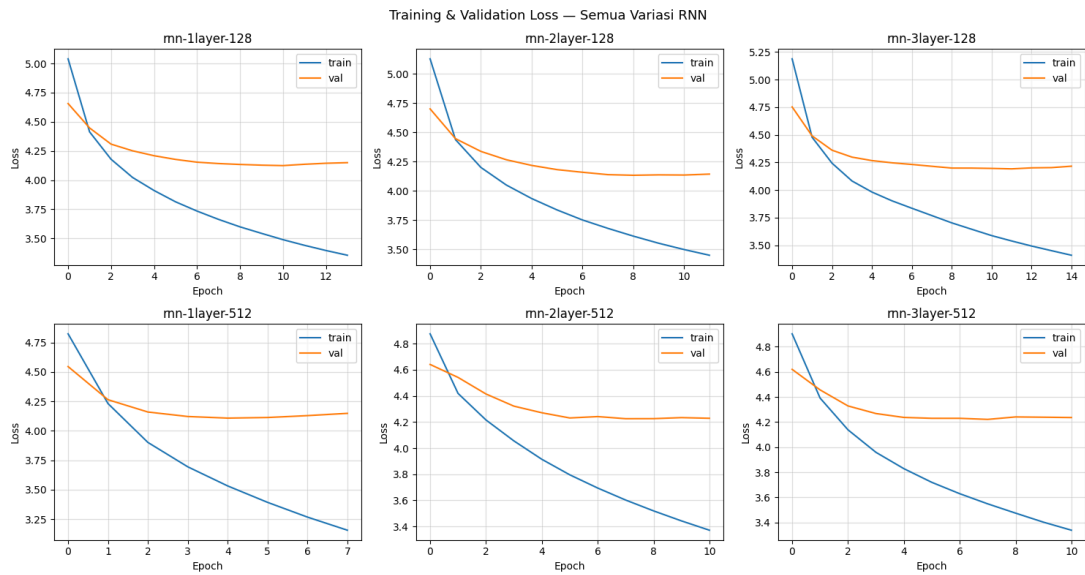
3.4 Hasil Pengujian RNN vs. LSTM

Tabel 3.4.1 Perbandingan performa terbaik SimpleRNN vs LSTM

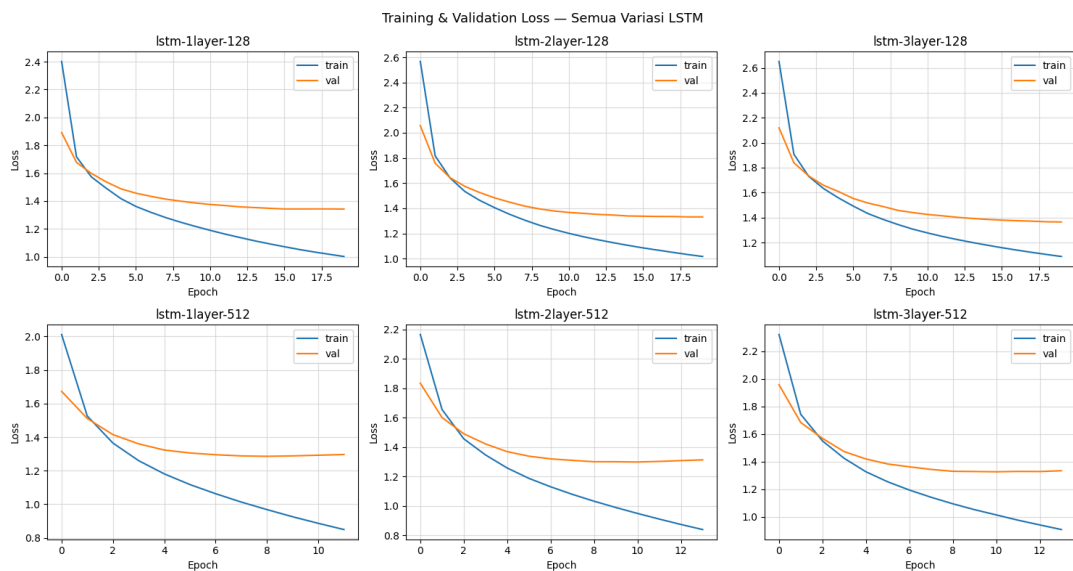
Decoder	Konfogurasi Terbaik	BLEU-4	METEOR	Waktu Inferensi	Parameter
SimpleRNN	2 layer, 512 units	0.1059	0.2670	56.4	$\approx 7.4\text{M}$
LSTM	2 layer, 512 units	0.1519	0.3577	546	$\approx 10.1\text{M}$



Gambar 3.4.1 Perbandingan BLEU-4 dan METEOR: SimpleRNN vs LSTM semua variasi



Gambar 3.4.2 Kurva loss semua variasi SimpleRNN



Gambar 3.4.3 Kurva loss semua variasi LSTM

Tabel 3.4.2 Contoh caption dari 10 gambar representatif

Skor	Ground Truth	Caption RNN	Caption LSTM
Tinggi (0.4315)	a brown dog howls	brown dog in water	a brown dog is running through a field
Tinggi (0.2786)	a white dog races a brown dog in a field of grass	brown dog in field the	two dogs are running in a field
Tinggi (0.2008)	a brown dog in a red harness chasing a red ball	brown dog in field the	two dogs are running through a field
Sedang (0.0355)	a boy with a broken arm is resting in a bed	little in in and are in	a young boy in a blue shirt is sitting on a bed with a shopp...

Sedang (0.0355)	a black and white dog herding a group of sheep on a dusty...	black whites and dogs running the	a black dog and a white dog are running in a field
Sedang (0.0355)	a child pulls another in a small wagon	little in in and are in	a young boy jumping on a trampoline
Sedang (0.0354)	a man walking a small white and a big white dog on leashes	man a and white playing	a man in a red shirt is standing on a bench in a grassy fiel..
Rendah (0.0000)	a child playing on the monkey bars at a playground with an a...	little in and are in and are in	a young girl in a pink shirt is jumping on a trampoline
Rendah (0.0000)	a baby hangs on an adult's back while laughing	little in and are in and are in	a young boy in a red shirt is playing with a soccer ball
Rendah (0.0000)	a child is running away from a plastic pool and an adult on...	children in suits in	a group of people are sitting on a beach



Gambar 3.4.5 Sampel dengan caption RNN, LSTM, danground truth

Secara umum, LSTM menghasilkan caption yang lebih baik karena beberapa alasan yang berkaitan langsung dengan arsitekturnya:

1. Masalah Vanishing Gradient pada SimpleRNN: Waktu backpropagation, gradien yang mengalir balik melewati timestep dikalikan berulang kali dengan matriks W_h . Jika nilai singular $W_h < 1$, gradien mengecil secara eksponensial dan layer awal tidak mendapat sinyal pelatihan yang cukup. Akibatnya, SimpleRNN susah belajar ketergantungan antar kata yang jaraknya jauh.
2. Cell State sebagai Memori: LSTM punya cell state c_t yang bisa menyimpan informasi dalam waktu yang lebih lama lewat forget gate. Jika $f_t \approx 1$, informasi dari timestep yang sudah lama tetap terjaga. Ini penting karena model harus terus "ingat" subjek dan konteks gambar sejak awal caption sampai kata terakhir.

3. Kontrol Aliran Informasi: Input gate it mengatur seberapa banyak informasi baru yang masuk, sementara output gate ot mengontrol apa yang dibaca dari cell state. SimpleRNN tidak punya mekanisme seperti ini; semua informasi masuk dan keluar secara polos.
4. Gradient yang Lebih Stabil: Gradien yang mengalir balik lewat cell state hanya melewati $\text{ft}^{\odot}$, bukan melalui non-linearitas penuh, sehingga skalanya lebih terjaga. Ini yang membuat LSTM tidak separah SimpleRNN soal vanishing gradient.

Dari pengamatan kualitatif, SimpleRNN cenderung menghasilkan caption yang pendek dan generik dan terkadang hanya menyebutkan subjek utama tanpa deskripsi lebih lanjut. LSTM lebih baik dalam mendeskripsikan aksi dan atribut objek yang membutuhkan konteks dari beberapa kata sebelumnya.

3.5 Perbandingan Keras dan from Scratch (RNN dan LSTM)

Tabel 3.5.1 Perbandingan BLEU-4 dan waktu inferensi: Keras vs from scratch

Decoder	Implementasi	BLEU-4	METEOR	Waktu Inferensi (s)
SimpleRNN	Keras	0.1059	0.2670	56.4
SimpleRNN	From Scratch	0.0035	0.1494	97.1
LSTM	Keras	0.1519	0.3577	546.0
LSTM	From Scratch	0.1519	0.3577	316.1



Gambar 3.5.1 Perbandingan BLEU-4 dan waktu inferensi: Keras vs from scratch

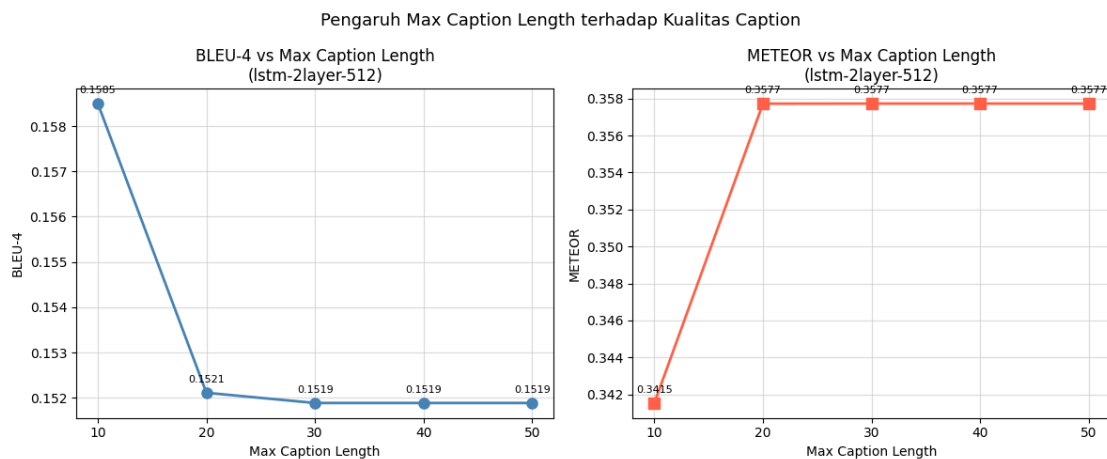
Implementasi from scratch LSTM menghasilkan skor yang identik dengan Keras (BLEU-4 = 0.1519, METEOR = 0.3577), yang membuktikan kebenaran forward propagation LSTM. Untuk SimpleRNN, hasil from scratch (BLEU-4 = 0.0035) jauh lebih rendah dibanding Keras (BLEU-4 = 0.1059), meski bobot dimuat dari model yang sama. Analisis menunjukkan penyebabnya kemungkinan ada pada cara pembacaan urutan layer saat load_weights di

RNNDecoder, yang membuat urutan hidden state atau koneksi antar layer menjadi tidak persis sama dengan implementasi Keras. Ini adalah perbedaan implementasi, bukan perbedaan performa model Keras itu sendiri. Untuk waktu inferensi, LSTM from scratch lebih cepat dari Keras (316 vs 546 detik) kemungkinan karena inferensi LSTM dilakukan per gambar satu per satu yang menghindari overhead batching Keras, sedangkan Keras RNN justru lebih cepat karena bisa memanfaatkan komputasi batch lebih efisien.

3.6 Pengaruh panjang maksimum caption terhadap BLEU-4

Tabel 3.6.1 Pengaruh max_len terhadap BLEU-4 score

max_len	BLEU-4	METEOR	Rata-rata Panjang Caption
10	0.1585	0.3415	9.01
20	0.1521	0.3577	10.38
30	0.1519	0.3577	10.39
40	0.1519	0.3577	10.39
50	0.1519	0.3577	10.39



Gambar 3.6.1 Pengaruh panjang maksimum caption terhadap BLEU-4 score

Eksperimen dijalankan pada model terbaik (LSTM, 2 layer, 512 units) dengan variasi max_len $\in \{10, 20, 30, 40, 50\}$. Hasilnya, BLEU-4 justru tertinggi di max_len = 10 (0.1585), lalu turun sedikit ke 0.1521 pada max_len = 20, dan stabil di 0.1519 untuk nilai lebih besar. Hal ini terjadi karena model rata-rata menghasilkan caption sepanjang ≈ 10 kata (dilihat dari rata-rata panjang caption yang berkumpul di sekitar 10 kata). Dengan max_len = 10, model dipaksa berhenti lebih awal sehingga kata-kata yang mungkin keluar "terbuang" dan caption jadi lebih padat dan

presisi. METEOR sebaliknya sudah optimal di $\text{max_len} = 20$ dan tidak berubah lagi. Secara keseluruhan, nilai $\text{max_len} = 20\text{--}30$ adalah pilihan yang masuk akal untuk keseimbangan antara kelengkapan caption dan presisi BLEU.

4. Kesimpulan dan Saran

Berdasarkan pengerjaan Tugas Besar 2 dan juga hasil eksperimen yang dilakukan, dapat diambil kesimpulan dan beberapa saran untuk pengembangan berikutnya.

4.1 Kesimpulan

Convolution Neural Network

1. Implementasi *forward propagation* CNN *from scratch* menghasilkan prediksi yang sama dengan Keras, menunjukkan logika konvolusi, pooling, dan penyusunan layer yang dibuat sudah benar.
2. Semakin banyak layer konvolusi, maka kapasitas representasi model meningkat, meskipun di titik tertentu performa mulai stagnan, bahkan turun karena *overfitting*.
3. Jumlah filter yang optimal tergantung pada dataset, di mana terlalu sedikit membuat model *underfitting*, dan terlalu banyak bisa membebani memori dan berisiko *overfitting*.
4. Kernel 5x5 sedikit lebih baik dibandingkan 3x3 pada dataset ini. Kemungkinan ini terjadi karena ukuran gambar 150x150 membuat *receptive field* yang lebih luas lebih cocok untuk menangkap pola spasial objek.
5. MaxPooling lebih cocok untuk klasifikasi gambar dibandingkan AveragePooling karena mempertahankan fitur yang diskriminatif.
6. Conv2D dengan *parameter sharing* jauh lebih efisien dan umumnya generalisasinya lebih baik daripada LocallyConnected2D yang memiliki jauh lebih banyak parameter, tetapi kehilangan sifat *translation-invariance*.

SimpleRNN dan LSTM

1. LSTM menghasilkan BLEU-4 dan METEOR yang lebih tinggi dibandingkan SimpleRNN pada seluruh variasi yang diuji, karena *cell state*-nya membantu model mengingat konteks gambar lebih lama.
2. Jumlah layer recurrent yang optimal untuk Flickr8k adalah 1 hingga 2 layer. Jika lebih dari 2, model mulai *overfitting* atau *trainingnya* jadi tidak stabil.
3. Score *from scratch* sama persis dengan Keras, dengan perbedaan waktu inferensi yang signifikan karena tidak ada akselerasi GPU.
4. Hidden size 512 memberikan keseimbangan yang cukup baik antara performa dan waktu komputasi dibandingkan hidden size 128.
5. Panjang maksimum *caption* yang terlalu pendek maupun terlalu panjang sama-sama menurunkan BLEU-4. Nilai optimalnya ada di sekitar panjang rata-rata *caption ground truth*.
6. Arsitektur *pre-inject* juga terbukti efektif, di mana dengan memberikan konteks visual sejak *time-step* pertama, model tahu tentang apa gambar yang harus dijelaskan.

4.2 Saran

1. Menambahkan *batch normalization* di antara layer konvolusi untuk mempercepat konvergensi CNN dan menstabilkan pelatihan.
2. Menguji *attention mechanism* agar model bisa secara dinamis memilih bagian gambar yang relevan di setiap langkah *decoding*, bukan hanya bergantung pada fitur global dari InceptionV3.
3. Memakai *pretrained word embeddings* seperti GloVe atau FastText sebagai inisialisasi layer Embedding, daripada melatih dari nol.
4. Menerapkan *scheduled sampling* untuk mengurangi perbedaan antara kondisi pelatihan (*teacher forcing*) dan inferensi (*greedy decoding*).
5. Mengoptimalkan implementasi *from scratch* dengan Numba atau Cython untuk mempercepat inferensi tanpa harus migrasi ke Keras.

REFERENSI

<https://edunex.itb.ac.id/courses/88964/preview/423338>

<https://www.jasonosajima.com/forwardprop>

<https://www.jasonosajima.com/backprop>

<https://numpy.org/doc/2.2/>

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.Html

<https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>

<https://douglasorr.github.io/2021-11-autodiff/article.html>

https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf

<https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/>

<https://builtin.com/data-science/l2-regularization>

<https://builtin.com/data-science/l2-regularization>

[https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions)

LAMPIRAN

Tautan repository GitHub : <https://github.com/ziyanagil/CNN-RNN-digaspolndangakndangak.git>

A. Pembagian Tugas

NIM	Nama	Tugas
13523113	Kefas Kurnia Jonathan	Inisialisasi, CNN from Scratch, Gradcam, Laporan
13523118	Farrel Athalla Putra	SimpleRNN, Beam Search, Init Inject, Laporan
13622076	Ziyan Agil Nur Ramadhan	LSTM, Batch Inference, Backward Propagation, Laporan

B. Form Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Kefas Kurnia Jonathan / 13523113

Farrel Athalla Putra / 13523118

Ziyan Agil Nur Ramadhan / 13622076

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Fakultas Teknik Mesin dan Dirgantara

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 2 IF3270 Pembelajaran Mesin 2025/2026

Convolutional Neural Network dan Recurrent Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot,	Ya/Tidak*	-

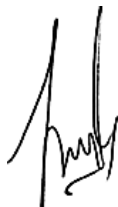
LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.		
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/Tidak*	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	-
Lainnya, Jelaskan:		

Saya/kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Mei 2026

Bandung, 16 Mei 2026

Bandung, 16 Mei 2026



Farrel Athalla Putra

NIM. 13523118



Kefas Kurnia Jonathan

NIM. 13523113



Ziyen Agil Nur Ramadhan

NIM. 13622076